STUDENT PERFORMANCE TREND ANALYSIS

Python Activity-Based Learning (ABL) Assignment — Sample Implementation

1. Problem Statement

Develop a Python program to compare student assessment scores across multiple examinations, calculate improvements and declines, identify consistently weak subjects, and classify students based on their overall average.

2. Objectives

• Store marks for multiple students, subjects, and examinations.

• Calculate total marks, overall average, and change between the first and final examinations.

• Identify subjects scoring below 50 in every examination.

• Classify students using clearly defined performance criteria.

• Display a readable report in the terminal.

3. Assumptions and Classification Rules

Assumptions

1. Each student has scores recorded for multiple subjects across multiple examinations.
2. Each subject score is measured out of 100 marks.
3. At least two examination scores are required to calculate improvement or decline.
4. Student records contain valid marks between 0 and 100.
5. A subject is considered weak if the student scores below 40 marks.
6. Performance trends are calculated by comparing the latest examination scores with the previous examination scores.

Classification Rules

1. Excellent: Average score is 80 or above.
2. Good: Average score is 60 or above but below 80.
3. Average: Average score is 40 or above but below 60.
4. Poor: Average score is below 40.

5. Improved Performance: Latest average score is greater than the previous examination's average score.
6. Declined Performance: Latest average score is less than the previous examination's average score.
7. No Change: Latest average score is equal to the previous examination's average score.
8. Consistently Weak Subject: A subject is considered consistently weak if the student scores below 40 in that subject in every examination.
9. Overall Performance: The student's classification is determined using the average score across all subjects in the latest examination.

4. Algorithm

1. Start and load the student marks for each subject across three examinations.

2. For each student, calculate totals for Exam 1, Exam 2, and the Final Exam.

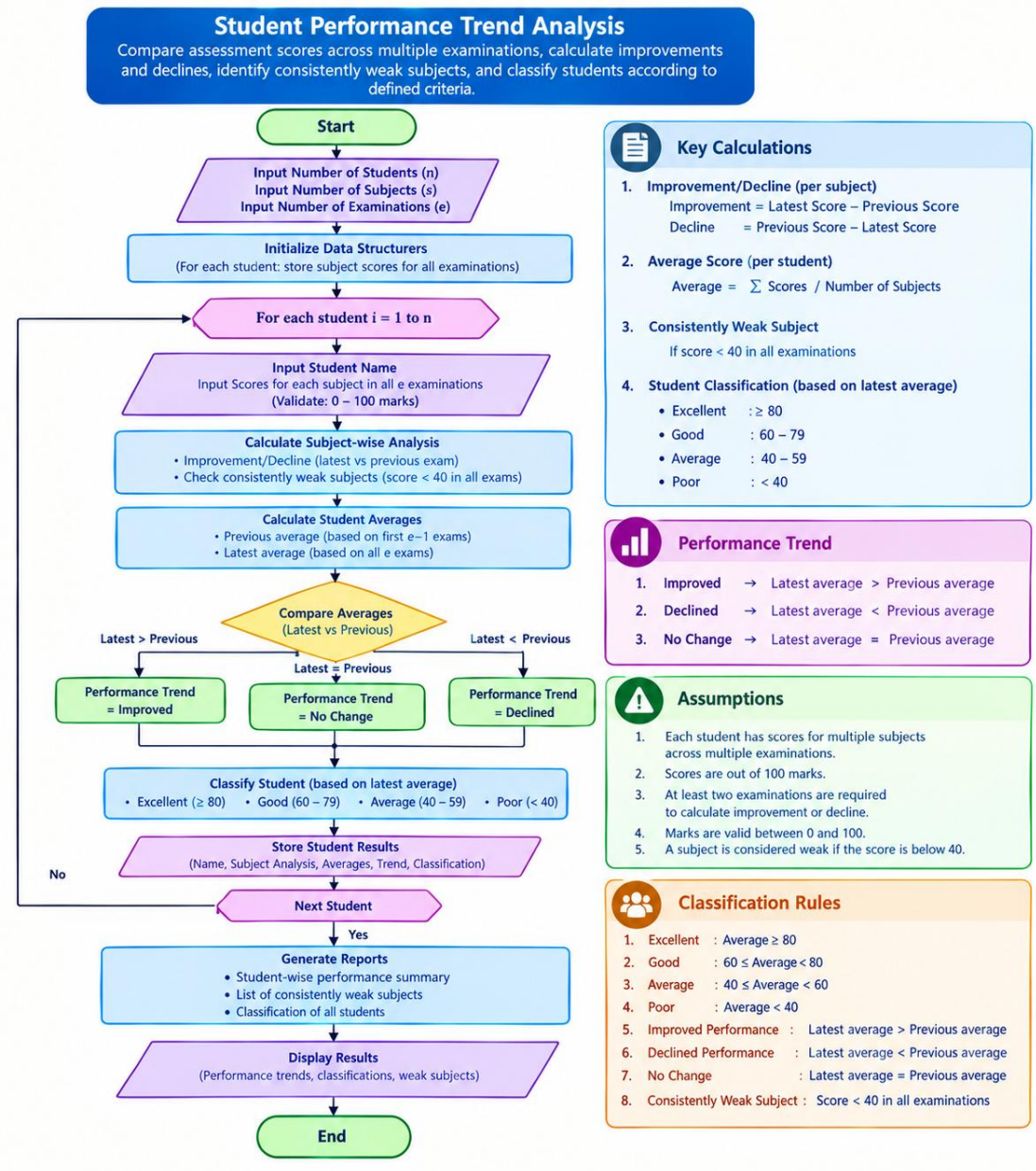
3. Calculate overall average and the change from Exam 1 total to Final Exam total.

4. Classify the student using the overall average.

5. For each subject, check whether every exam score is below 50.

6. Print the summary table and subject-wise trends.

5. Flowchart


Student Performance Trend Analysis
Compare assessment scores across multiple examinations, calculate improvements and declines, identify consistently weak subjects, and classify students according to defined criteria.
Start
Input Number of Students (n)
Input Number of Subjects (s)
Input Number of Examinations (e)
Initialize Data Structurers
(For each student: store subject scores for all examinations)
For each student i = 1 to n
Input Student Name
Input Scores for each subject in all e examinations
(Validate: 0 – 100 marks)
Calculate Subject-wise Analysis
• Improvement/Decline (latest vs previous exam)
• Check consistently weak subjects (score < 40 in all exams)
Calculate Student Averages
• Previous average (based on first e−1 exams)
• Latest average (based on all e exams)
Compare Averages
(Latest vs Previous)
Latest > Previous
Latest = Previous
Latest < Previous
Performance Trend = Improved
Performance Trend = No Change
Performance Trend = Declined
Classify Student (based on latest average)
• Excellent (≥ 80) • Good (60 – 79) • Average (40 – 59) • Poor (< 40)
Store Student Results
(Name, Subject Analysis, Averages, Trend, Classification)
No
Next Student
Yes
Generate Reports
• Student-wise performance summary
• List of consistently weak subjects
• Classification of all students
Display Results
(Performance trends, classifications, weak subjects)
End
Key Calculations
1. Improvement/Decline (per subject)
Improvement = Latest Score – Previous Score
Decline = Previous Score – Latest Score
2. Average Score (per student)
Average = ∑ Scores / Number of Subjects
3. Consistently Weak Subject
If score < 40 in all examinations
4. Student Classification (based on latest average)
• Excellent : ≥ 80
• Good : 60 – 79
• Average : 40 – 59
• Poor : < 40
Performance Trend
1. Improved → Latest average > Previous average
2. Declined → Latest average < Previous average
3. No Change → Latest average = Previous average
Assumptions
1. Each student has scores for multiple subjects across multiple examinations.
2. Scores are out of 100 marks.
3. At least two examinations are required to calculate improvement or decline.
4. Marks are valid between 0 and 100.
5. A subject is considered weak if the score is below 40.
Classification Rules
1. Excellent : Average ≥ 80
2. Good : 60 ≤ Average < 80
3. Average : 40 ≤ Average < 60
4. Poor : Average < 40
5. Improved Performance : Latest average > Previous average
6. Declined Performance : Latest average < Previous average
7. No Change : Latest average = Previous average
8. Consistently Weak Subject : Score < 40 in all examinations

6. Pseudocode

```
BEGIN

  INPUT number_of_students
  INPUT number_of_subjects
  INPUT number_of_examinations

  FOR each student from 1 to number_of_students DO

    INPUT student_name

    FOR each subject DO
      FOR each examination DO
        INPUT score

        WHILE score < 0 OR score > 100 DO
          DISPLAY "Invalid score. Enter marks between 0 and 100"
          INPUT score
        END WHILE

        STORE score
      END FOR
    END FOR

    SET total_latest = 0
    SET total_previous = 0
```

```
SET weak_subject_count = 0

FOR each subject DO

  SET previous_score = score in previous examination
  SET latest_score = score in latest examination

  SET improvement = latest_score - previous_score

  IF improvement > 0 THEN
    DISPLAY subject, "Performance Improved"
  ELSE IF improvement < 0 THEN
    DISPLAY subject, "Performance Declined"
  ELSE
    DISPLAY subject, "No Change"
  END IF

  IF scores in all examinations < 40 THEN
    DISPLAY subject, "Consistently Weak"
    INCREMENT weak_subject_count
  END IF

  ADD latest_score TO total_latest
  ADD previous_score TO total_previous

END FOR
```

```
SET latest_average = total_latest / number_of_subjects
SET previous_average = total_previous / number_of_subjects

IF latest_average > previous_average THEN
  SET performance_trend = "Improved"
ELSE IF latest_average < previous_average THEN
  SET performance_trend = "Declined"
ELSE
  SET performance_trend = "No Change"
END IF

IF latest_average >= 80 THEN
  SET classification = "Excellent"
ELSE IF latest_average >= 60 THEN
  SET classification = "Good"
ELSE IF latest_average >= 40 THEN
  SET classification = "Average"
ELSE
  SET classification = "Poor"
END IF

DISPLAY student_name
DISPLAY previous_average
DISPLAY latest_average
DISPLAY performance_trend
```

```
        DISPLAY weak_subject_count
        DISPLAY classification

    END FOR

    GENERATE student performance report
    DISPLAY final report

END
```

7. Python Source Code

```
# Student Performance Trend Analysis

def analyze_student():
    students = []

    n = int(input("Enter number of students: "))
    subjects_count = int(input("Enter number of subjects: "))
    exams_count = int(input("Enter number of examinations (minimum 2): "))

    if n <= 0 or subjects_count <= 0 or exams_count < 2:
        print("Invalid input. Enter positive counts and at least 2 examinations.")
        return

    subjects = []
    for i in range(subjects_count):
```

```
    subject = input(f"Enter subject {i + 1} name: ")
    subjects.append(subject)

for i in range(n):
    print(f"\n--- Student {i + 1} ---")
    name = input("Enter student name: ")
    scores = {}

    for subject in subjects:
        scores[subject] = []
        for exam in range(exams_count):
            while True:
                try:
                    mark = float(input(
                        f"{subject} - Exam {exam + 1} score (0-100): "
                    ))
                    if 0 <= mark <= 100:
                        scores[subject].append(mark)
                        break
                    print("Invalid score! Enter marks between 0 and 100.")
                except ValueError:
                    print("Please enter a valid number.")

    previous_total = sum(scores[s][-2] for s in subjects)
    latest_total = sum(scores[s][-1] for s in subjects)
```

```
previous_avg = previous_total / subjects_count
latest_avg = latest_total / subjects_count

print(f"\nPerformance Report for {name}")
print("-" * 65)
print(f"{'Subject':<15}{'Previous':<12}{'Latest':<12}"
    f"{'Change':<12}{'Trend':<15}")
print("-" * 65)

weak_subjects = []

for subject in subjects:
   previous = scores[subject][-2]
   latest = scores[subject][-1]
   change = latest - previous

   if change > 0:
      trend = "Improved"
   elif change < 0:
      trend = "Declined"
   else:
      trend = "No Change"

   if all(mark < 40 for mark in scores[subject]):
      weak_subjects.append(subject)
```

```
    print(f"{subject:<15}{previous:<12.1f}{latest:<12.1f}"
          f"{change:+.1f}{'':<7}{trend:<15}")

if latest_avg >= 80:
    classification = "Excellent"
elif latest_avg >= 60:
    classification = "Good"
elif latest_avg >= 40:
    classification = "Average"
else:
    classification = "Poor"

if latest_avg > previous_avg:
    overall_trend = "Improved"
elif latest_avg < previous_avg:
    overall_trend = "Declined"
else:
    overall_trend = "No Change"

print("-" * 65)
print(f"Previous Exam Average : {previous_avg:.2f}")
print(f"Latest Exam Average   : {latest_avg:.2f}")
print(f"Overall Performance   : {overall_trend}")
print(f"Classification        : {classification}")

if weak_subjects:
```

```
            print("Consistently Weak Subjects:", ", ".join(weak_subjects))
        else:
            print("Consistently Weak Subjects: None")

        students.append({
            "name": name,
            "average": latest_avg,
            "trend": overall_trend,
            "classification": classification
        })

    print("\n========== FINAL STUDENT SUMMARY ==========")
    print(f"{'Student':<20}{'Average':<12}{'Trend':<15}{'Class':<12}")
    print("-" * 59)

    for student in students:
        print(f"{student['name']:<20}{student['average']:<12.2f}"
              f"{student['trend']:<15}{student['classification']:<12}")

    print("\nAnalysis completed successfully.")


analyze_student()
```

8. Output Screenshot

```
Enter number of students: 2
Enter number of subjects: 3
Enter number of examinations (minimum 2): 3
Enter subject 1 name: Mathematics
Enter subject 2 name: Science
Enter subject 3 name: English

--- Student 1 ---
Enter student name: Lekhana
Mathematics - Exam 1 score (0-100): 60
Mathematics - Exam 2 score (0-100): 70
Mathematics - Exam 3 score (0-100): 85
Science - Exam 1 score (0-100): 35
Science - Exam 2 score (0-100): 38
Science - Exam 3 score (0-100): 39
English - Exam 1 score (0-100): 70
English - Exam 2 score (0-100): 75
English - Exam 3 score (0-100): 80

Performance Report for Lekhana
------------------------------------------------------------------
Subject         Previous    Latest      Change      Trend
------------------------------------------------------------------
Mathematics     70.0        85.0        +15.0        Improved
Science         38.0        39.0        +1.0        Improved
English         75.0        80.0        +5.0        Improved
------------------------------------------------------------------
Previous Exam Average : 61.00
Latest Exam Average   : 68.00
Overall Performance   : Improved
Classification        : Good
Consistently Weak Subjects: Science

Performance Report for Nikhitha
------------------------------------------------------------------
Subject         Previous    Latest      Change      Trend
------------------------------------------------------------------
Mathematics     80.0        75.0        -5.0        Declined
Science         85.0        80.0        -5.0        Declined
English         70.0        65.0        -5.0        Declined
------------------------------------------------------------------
Previous Exam Average : 78.33
Latest Exam Average   : 73.33
Overall Performance   : Declined
Classification        : Good
Consistently Weak Subjects: None

========== FINAL STUDENT SUMMARY ==========
Student             Average     Trend          Class
-----------------------------------------------------------
Lekhana             68.00       Improved       Good
Nikhitha            73.33       Declined       Good

Analysis completed successfully.
```

9. Test Cases

1. **Valid Student Input:** Verify that the system accepts valid student names, subjects, and examination scores.

2. **Multiple Examination Scores:** Check whether the system stores and compares scores across multiple examinations correctly.

3. **Performance Improvement:** Verify that the system identifies improvement when the latest examination score is higher than the previous examination score.

4. **Performance Decline:** Check whether the system identifies a decline when the latest examination score is lower than the previous examination score.

5. **No Change in Performance:** Verify that the system displays “No Change” when the latest and previous examination scores are equal.

6. **Average Score Calculation:** Check whether the system calculates the average marks correctly for each student.

7. **Excellent Classification:** Verify that students with an average score of 80 or above are classified as Excellent.

8. **Good Classification:** Check whether students with an average score of 60 or above but below 80 are classified as Good.

9. **Average Classification:** Verify that students with an average score of 40 or above but below 60 are classified as Average.

10. **Poor Classification:** Check whether students with an average score below 40 are classified as Poor.

11. **Consistently Weak Subjects:** Verify that a subject is identified as consistently weak when the student scores below 40 in every examination.

12. **Invalid Marks:** Check whether the system rejects marks below 0 or above 100.

13. **Invalid Input:** Verify that the system handles non-numeric examination scores appropriately.

14. **Minimum Examination Validation:** Check whether the system requires at least two examinations to calculate performance trends.

15. **Final Student Summary:** Verify that the final report displays each student's latest average, overall trend, and performance classification correctly.

10. Conclusion

The program summarizes assessment trends, quantifies score changes, flags subjects that remain below the chosen threshold, and assigns performance categories. The marks and thresholds are illustrative; replace them with your faculty's required rules if different.